



Internal Revenue Service

IRS Strong Authentication System for Application-to-Application Web Services

Version 1.3

August 8, 2017

Document History

Revision History

Version Number	Date	Summary of Changes	Changes Marked
1.0		Initial draft	No
1.1	11/15/2011	Revised version	No
1.2	06/16/2014	Updated referenced URLs Updated Digital certificate information in section 3.6	No
1.3	8/8/2017	Updated Section 3.6.5, removed ORC business representative certificate	No

Table of Contents

1. Introduction.....	1
1.1 Background.....	1
1.2 The Purpose of the Code.....	1
1.3 The Scope of the Feature	2
1.4 The Purpose of This Document	2
1.5 Document Organization.....	3
2. Software Design Considerations.....	4
2.1 Overview of Brokered Client Authentication.....	4
2.1.1 Authentication Methods.....	5
2.1.2 Overview of Automated Mechanisms for Verifying Identity.....	5
2.2 Overview of the Client Code	6
2.2.1 Strong Authentication Mechanism	6
2.2.2 Steps in the Message Transfer Process	6
2.3 Specification of Strong Authentication Requirements	7
2.4 Client Design Considerations	9
2.5 Client Integration	9
3. Incorporating the Signing Code	10
3.1 Overview.....	10
3.2 Coding Conventions.....	10
3.3 Java Source	11
3.3.1 Class Description	11
3.3.2 Fields and Methods.....	11
3.3.3 Exceptions.....	12
3.3.4 Integrating the Code.....	12
3.4 Visual Basic .Net Source	13
3.4.1 Class Description	13
3.4.2 Fields and Methods.....	13
3.4.3 Integrating the Code.....	13
3.5 Design Considerations	14
3.5.1 Java Design Considerations	14
3.5.2 .Net Design Considerations	14

3.6	Authorized Certificate Authorities.....	14
3.6.1	Entrust	14
3.6.2	Identrust	14
3.6.3	Verisign.....	15
3.6.4	Treasury Enterprise Certificate Authority	15
3.6.5	ORC	15
Appendix A. The Signing Code		16
A.1	The Java Source Code.....	16
A.2	The .Net Source Code	26
Acronyms		33
Glossary		34
References		35

1. Introduction

This document describes a set of modules that will be used to perform authentication tasks when registered third-party users connect to the Internal Revenue Service (IRS) portal through the Application-to-Application (A2A) interface. This authentication method will be required at some future date for all externals using the A2A interface for the IRS's Modernized eFile (MeF) program and other A2A applications. The mechanisms described in this document are intended only to guide the initial development of application authentication mechanisms performing an authorized login to the IRS's portal through the A2A interface. Other uses of this guide or the accompanying software are not authorized.

1.1 Background

Early in 2005, the IRS determined that a need existed for additional authentication security for A2A transfers. This need was driven by a policy requirement received from the U.S. Department of the Treasury. According to the Department's policy statement,

Inbound services shall be prohibited unless a valid business case can establish their validity. Inbound services shall provide strong authentication using one-time or session passwords, challenge and response protocols, digital signatures, or encryption.

In June 2005, via the Tax Information Group for Electronic Data Interchange Requirements Standardization (TIGERS) working group, the IRS began a dialog with organizations external to the IRS in order to identify preferred options for next-generation authentication.

In September 2005, TIGERS forwarded recommendations to IRS on authentication options, identifying a preference for Digital Certificate authentication.

In early 2006, the IRS prototyped a Web Services (WS) Security-based certificate option in the laboratory prior to making a decision regarding final implementation. Upon successful completion of the prototype, the IRS discussed the option with externals and ultimately decided to deploy the service in production during 2007.

This option uses certificates to start the session under the Web Services Security Services standard. As a result, passwords will be eliminated from the logon process and A2A will rely on stronger public key methods for authentication. Although this option provides much stronger security for A2A transactions, it also necessitates changes in pre-existing client code. During prototyping, the IRS worked with two IRS customers, both of which provided actual MeF client software for testing purposes, to develop and test new client signing code for both JAVA and .NET clients. The client changes included in this document were tested successfully using those applications.

1.2 The Purpose of the Code

The IRS has implemented a solution to mitigate security vulnerabilities involved in Web service requests via the A2A interface. Since this portal is a primary means of transferring data in support of the MeF program, the IRS worked with states and transmitters to define stringent authentication criteria for those transfers. Since the transfer mechanism involved unattended transfers, the solution was also required to support an automated system for controlling access to

sensitive data. Consequently, the new system requires each external entity to digitally sign authentication requests via the A2A interface. Digitally signing messages is not a commonly understood process, so The MITRE Corporation, under IRS sponsorship, developed a prototype of the signing mechanism using Java and Visual Basic. This code was developed as a prototype to demonstrate the technique, using standard MeF client software, and to provide a tested example for digitally signing selected Simple Object Access Protocol (SOAP) messages prior to delivery to the A2A portal. The code has been released utilizing a standard open source license to enable wide distribution incorporating open source components.

The solution, as deployed, requires integration with local client code at each external entity. All externals implementing this code must evaluate the most appropriate means of integrating the functionality into their existing systems. The IRS does not require use of this code if an alternate signing process can be developed that is compatible with the current portal system.

The signing code described in this guide creates a digital signature by which certain A2A client applications can authenticate to IRS Portal-based Web service applications.

NOTICE: This code has been provided as an example of a working client stub. It is not intended to serve as production code and is offered without warranty for any purpose. Please see the distribution notice included in the source for your particular environment.

1.3 The Scope of the Feature

To support strong authentication for A2A Web services, the IRS is installing certificate-based authentication technology. The change to the IRS code described above applies to IRS A2A Web services processing only. That processing currently uses Web Services to interact with the portal. The new authentication system will affect authentication for all A2A Web services.

This change will not affect electronic management system (EMS) or MeF Internet browser-based transfers (Internet Filing Application [IFA]).

1.4 The Purpose of This Document

This document is intended to explain the integration and use of this IRS-provided client code sample to support certificate-based authentication for A2A Web services. In addition to the code itself, it provides necessary information and guidance that developers may use when integrating the new feature into client software that communicates with the IRS A2A Web services. This guidance is relevant both to client code that will be developed in the future and to pre-existing client code not currently using digital signatures to sign SOAP messages.

1.5 Document Organization

The remainder of this document is organized as follows:

Section	Purpose
Section 2: Software Design Considerations	Provides an overview of the client code and describes the requirements that the code has been designed to satisfy
Section 3: Incorporating the Signing Code	Describes two methods for implementing the code and presents design considerations for each
Appendix A: The Signing Code	Source code for modules described in this document
Acronyms	Lists and defines the acronyms used in this document
Glossary	Explains key terms used in this document
References	Lists references to relevant regulations and code standards

2. Software Design Considerations

The software provided for strong authentication support was designed to meet specific requirements established by the Strong Authentication Working Group without adversely impacting ongoing operations of the IRS portal infrastructure. Although many designs were considered, the resulting design is considered to be effective and advantageous for the largest number of externals. The design changes only one step in the login process for applications using the portal, a change that strengthens authentication processes while eliminating passwords.

2.1 Overview of Brokered Client Authentication

[The Information Assurance Technical Framework (IATF) Forum, sponsored by the U.S. Government, provides an excellent overview of brokered client authentication using digital certificates. The remainder of Section 2.1 is extracted from IATF Release 3.1 (September 2002), Chapter 4, Technical Security Countermeasures, Subsection 4.3.1.1, I&A.]

The first step of most security services is to determine the identities of one or more of the parties participating in an action. A trusted identity must be used for access control decisions as well as to provide non-repudiation and accountability. Knowing the identity of a peer is also fundamental to establishing communication with confidentiality or integrity. If the identity of the peer in a secure communications path is not properly established, it may be possible that an unauthorized user (an adversary) could masquerade as an authorized user, leaving the data open to possible disclosure or manipulation by the adversary.

A mechanism must be in place to provide some assurance in the assignment of an identity. The entity that assigns identity must have a position with some level of trust (either implied or assured by another entity with a higher position or level of trust). These trusted entities must implement a process of identity checking that protects against assigning improper identities. Process examples include checking driver licenses or verifying fingerprints. Note that assigning identity is the equivalent of a registration process and that this process could take place using an existing security mechanism with its own identity establishment mechanism.

An identity must be unique across the community that will be validating that identity. This requires implementation of a community-wide assignment mechanism that provides a unique identity to each entity. An identity must also be conveyed in a representation that is recognized and able to be processed by all potential peers to a communication. This implies a standard format for representing identity.

The assurance level attributed to an identity depends on the processes used to verify the correctness of that identity before issuing it, the trust instilled by the entity assigning the identity, and the strength of the binding between the entity and the identity. Examples of verification range from requesting a mother's maiden name over the phone to checking driver licenses or verifying fingerprints in person. Examples of trust in issuers include procedural mechanisms such as a company's assigning system administrators, legal mechanisms such as notaries, or mechanisms provided by technology such as certification paths in a certification hierarchy. Examples of mechanisms for binding entities to identities include signed X.509 certificates and password files.

Strongly establishing identities for communicating entities is the first step in being able to counter any attack that is predicated on adversaries representing themselves as someone or something they are not. This includes masquerading and insider modification attacks.

2.1.1 Authentication Methods

To authenticate an entity attempting an access, an identity must be associated with the access request and provided to the communicating peer. Along with an indication of identity, the authenticating peer must have the additional parameters (authentication information) used to validate that identity. Authentication applications typically fall into two major categories: bilateral authentication including user-to-host and peer-to-peer architectures, and trusted third party (TTP).

User-to-Host Authentication – When a user begins a session with a host or workstation, the user needs to be identified and authenticated before access to the host or network is granted. This requires an authentication mechanism that authenticates a real person to a machine. The best methods require multiple forms of authentication such as password, physical token and biometric verification.

Trusted Third Party Authentication – The architecture for TTP authentication includes a third entity, trusted by all entities, to provide authentication information. A TTP may provide authentication information in each instance of authentication, in real-time, or as a precursor to an exchange (such as a Certificate Authority [CA]). The amount of trust given the third party must be evaluated. Methods for establishing and maintaining a level of trust in a TTP include Certification Practice Statements that establish rules, processes, and procedures that a CA uses to ensure the integrity of the authentication process and the use of secure protocols to interface with authentication servers.

The IRS has chosen to identify a partner entity using the processes of any of four trusted CAs, and then to enroll specific IRS partner Web service clients with a certificate issued by one of those CAs. See Section 3.6 for details.

2.1.2 Overview of Automated Mechanisms for Verifying Identity

There are a number of mechanisms for implementing and distributing identity and authentication information. These mechanisms include:

- Names and passwords stored in a database local to the entity making the access control decision
- IP addresses provided by a secure Domain Name Server (DNS)
- Passwords generated locally based on time (one-time passwords)
- Symmetric keys stored in a local database
- Public keys stored in a local database
- Public key certificates provided by directories in response to queries
- Authentication information carried in the communications protocols themselves

The assurance level of the communication of identity and authentication information processes depends on whether that information needs protecting and how well it is protected. For example,

passwords are sensitive because they can be used by anyone who knows them – they should be encrypted for storage and transport. Certificates can be stored in unprotected directories or carried on unencrypted communications channels because they can only be used by the entity that holds the associated private key.

There are entire infrastructures devoted to providing identities and a means of authenticating those identities. Examples of infrastructures supporting the determination of an authentic identity include the X.509 authentication framework, the Internet Engineering Task Force (IETF) Public Key Infrastructure (PKI), the secure DNS initiatives, and the Simple Public Key Infrastructure (SPKI).

The sections below describe the specific mechanism used to authenticate accounts under the revised IRS A2A Web services Strong Authentication system.

2.2 Overview of the Client Code

2.2.1 Strong Authentication Mechanism

The mechanism used by the system described in this document is based on the X.509 public key framework, operating within a Web Services environment. This type of brokered authentication uses digital certificates and X.509 digital signing to create a strong and standard method of identity verification. By relying on the cryptographic strength of the public key framework, users can eliminate many of the vulnerabilities inherent in password-based schemes.

The process implemented in the IRS A2A Web service Strong Authentication client code is well documented in various locations around the Internet. For a complete description of the method, readers should check the following references:

Java Signing Implementation:

<http://ws.apache.org/wss4j/index.html>

Microsoft Brokered Authentication with Message Signing:

<http://msdn.microsoft.com/library/default.asp?url=/library/en-us/wse/html/9421f0df-ccdd-4d67-9e85-1d102ada287e.asp>

Web Services Security for Java by Bilal Siddiqui on O'Reilly XML.COM:

<http://webservices.xml.com/pub/a/ws/2003/10/28/jwss.html>

The IRS has implemented the server side of the transaction using a commercial appliance. For this reason, clients should be coded as closely to the standard implementation as feasible to avoid potential interface problems. External entities will be notified when the current service is upgraded, either to accommodate a new version of Web Services Security or a new version of the standard signing process.

2.2.2 Steps in the Message Transfer Process

After installation of the client update, only one step will differ from the current six-step logon process:

Step 1: Client establishes a Secure Socket Layer (SSL) session with the IRS portal using IRS Server-side certificates. Transport Layer Security 1.0 protocols are preferred.

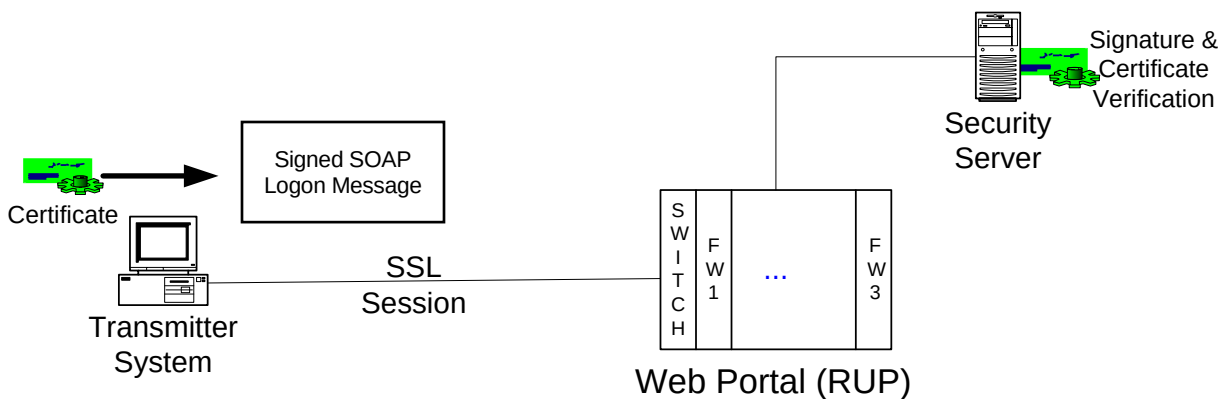
Step 2: Client creates a login request (SOAP) message including the certificate which was registered earlier for that client's ASID, using IRS Automated Enrollment.

Step 3: Client signs the SOAP message using a registered private key (rather than, as previously, including a password in the SOAP message header).

Step 4: Client sends the SOAP login request to the IRS A2A login URL.

Step 5: Server responds with a session token in the response (or returns a SOAP fault response with error code).

Step 6: Client makes other Web Services requests normally.



As shown above, clients sign the logon SOAP request instead of inserting a password into the SOAP header. By removing the password from the SOAP header, clients eliminate a threat to the integrity of the current return transfer mechanism. The Application System ID (ASID) is still required in the headers.

To establish a session, only the first message of the session (usually a login) must be signed. Alternately, for single SOAP message sessions, the client may sign a transactional SOAP message in the same way that clients could previously submit a transactional SOAP message with ASID and password loaded into the SOAP message header. The signed message will include the registered certificate and the ASID of the client.

When the server receives a properly signed message request, it will verify the signature and establish a session on behalf of the owning ASID client for transactions with the A2A system. This process is not available for browser-based interaction with MeF in the IRS portal.

2.3 Specification of Strong Authentication Requirements

To improve developer understanding of the provided code, this section includes requirements originally issued by the IRS for development of the Strong Authentication service. Although these requirements are not mandatory for installation and use of the code, they may be helpful when integrating the code into an existing A2A client application.

The IRS shall provide the capability to use client digital (X.509) certificates issued by IRS-designated certificate authorities (see Section 3.6) to provide enhanced authentication of external

partner application systems for A2A Web service processing. This capability is as an alternative to, and will eventually be a replacement for, current A2A password authentication mechanisms, and is intended to mitigate threats defined in Treasury Department Publication 85-01, Department of the Treasury Information Technology (IT) Security Program, Vol. II, Handbook, Part 1, Sensitive Systems, section 5.4.4.

To meet the requirements stated in the Treasury directive, the system will:

1. Provide verification of a digital signature of appropriate message parts produced using the private key for the X.509 certificate submitted with the authentication request, and shall prevent (fail) authentication if this verification fails.
2. Verify that a submitted X.509 certificate is properly formatted and is verified by appropriate CA signature, and shall prevent (fail) authentication if this verification fails.
3. Provide checks to ascertain that a submitted X.509 certificate has not been revoked, using most current Certificate Revocation List (CRL) resources from IRS-designated CAs, and shall prevent (fail) authentication if the certificate has been revoked. CRLs will be retrieved from each IRS-designated CA based on the schedule of available updates for that CA, so that the most up-to-date CRL will be made available for A2A authentication.
4. Provide checks to ensure that a submitted X.509 certificate has not expired, and shall prevent (fail) authentication if the certificate has expired.
5. Ensure that a submitted X.509 certificate matches a certificate previously registered for the application system using data stored in the IRS directory by an enrollment process, and shall prevent (fail) authentication if the match cannot be made.
6. Mitigate replay attacks for authentication.
7. Provide error handling by which any external failing strong authentication for any reason will receive a SOAP fault response message stating that “Authentication Failed.”
8. Provide adequate capacity for A2A certificate-based authentication.
9. Process 99 percent of A2A authentication requests within 4 seconds of receipt of the request.
10. Ensure that authentication attempts and results are audited and incorporated into log files for audit and forensic purposes.
11. Include a message in the SOAP response to a successful authentication request when the certificate used to authenticate the application system is within 30 days of expiration, stating the number of days to expiration. This note should be similar to the response note used to inform that a password is close to expiration.
12. Accommodate external users that have one or more sites (and client certificates) associated with a single PKI sponsor.
13. Register valid certificates for each enrolled external application system using an automated enrollment process. The authentication credentials structure shall be changed for all A2A Web service requests to accommodate session-less requests and requests where the first business request is used to log on and establish an authentication session.

Authorized externals will have the ability, as part of the automated enrollment application, to replace the certificate on file for an application system for reasons of revocation, compromise, or imminent expiration.

The system shall mitigate replay attacks. As part of the message verification process, the system shall validate the timestamp and (for MeF) Message Identifier inside the Application Header in a way that ensures uniqueness during the lifetime of a client session. The system shall prevent authentication messages from being intercepted, copied, and resubmitted.

Authenticated application systems shall be granted access to A2A Web service URLs for which they are authorized, and the client certificate identifier or identifiers shall be included in the log entry for the authentication event.

2.4 Client Design Considerations

When developing client code in either a Java or a .NET environment, it is important that developers generate code stubs using their application's Strong Authentication Web Services Design Language (WSDL). Clients are designed to generate a SOAP message and then send it to the server-side application. In the logic flow of the code, the call to the signing procedure is made after creating the SOAP message but before sending the message to the server.

The Java version of the sample signing code only accepts SOAP messages created using the Apache axis package; `javax.xml.soap.SOAPMessage` is not compatible with the signing code. If a client is developed using `javax.xml.soap.SOAPMessage` instead of `org.apache.axis.Message`, a workaround can be used to write the `SOAPMessage` to a file and then recreate the message using the appropriate type before sending it to the signing procedure. Developing the client code using the package from Apache would prevent any tampering with the XML while using the workaround.

2.5 Client Integration

Integrating the signature code requires certain changes in operations. Each time a certificate is purchased (see the Certificate Authorities section below) the certificate must be registered with the IRS. Registration is necessary to ensure that the specific ASID/certificate combination in use is authorized for A2A Web service processing. This also provides a certain amount of error control prior to production use of a certificate.

To register a new certificate for a client ASID, have your delegate log in and access the automated enrollment application, uploading a certificate file. (See the *Automated Enrollment User Guide*.) Certificate private keys must also be encrypted on the local machine. This means that implementers should create a password for the local KeyStore. The password will be used to open the KeyStore for operation. Implementers should consider disk encryption or other means to protect KeyStore password.

In some cases, clients will create files on the local system to store the SOAP message in preparation for signing. If that is the case, the signing routine must have write access to the file folder or directory in order to update the file in storage.

3. Incorporating the Signing Code

This code is intended to be a single function or class inside a larger A2A client program. Therefore, this document provides general guidance for integrating the code into existing applications. Two common coding examples are provided for the convenience of developers.

Developers wishing to use this code are advised to become familiar with appropriate signing mechanisms provided by various organizations in support of Web Services Authentication. (References are provided at the end of this document.) Additionally, developers should understand the security implications of these modules prior to making changes that could affect the integrity of the authentication process.

3.1 Overview

Two versions of the signing code were developed in order to accommodate the two most popular Web service development environments. The Java version was developed using j2sdk1.4_13 and should be compatible with newer development environments. The .Net version was developed in Visual Studio 2003 using Visual Basic. The .Net version is interoperable with all .Net languages and should be compatible with newer versions of Visual Studio.

3.2 Coding Conventions

Classes:

Each class in the Java and the .NET version of the code is preceded by a comment in javadoc style describing the purpose of the class and providing usage instruction, including any external packages or files that are needed. The following tags are also used:

@author author-name

@version version number of the class

Methods:

All methods (both versions) are preceded by a comment in javadoc style describing the purpose and preconditions and providing usage instructions. The following tags are also used:

@param paramName followed by a description

@return description of the return value

Naming Conventions:

Classes: CapitalizedWithInternalWordsCapitalized

Methods: firstWordLowerCaseButInternalWordsCapitalized

Variables: firstWordLowerCaseButInternalWordsCapitalized

Inline Comments:

Inline comments precede the lines they reference to help add additional information about what that particular line is doing.

3.3 Java Source

3.3.1 Class Description

The Java source implementation, `FS_Msg_Signer` class, contains methods that allow an unsigned message to be passed into the signer; a signed message is then returned. Key information and signing part information can be passed into the constructor when a new instance is created.

This implementation was developed using JDK1.4.2_13 from examples found at <http://ws.apache.org/wss4j/api.html>. A list of its detailed dependencies may be found in section 3.3.4.

Certificates are held in a Java keystore. Information about the keystore and how the `FS_Msg_Signer` should access it is found in a `crypto.properties` file, which is generally formatted as follows:

```
org.apache.ws.security.crypto.provider=org.apache.ws.security.components.crypto.Merlin
org.apache.ws.security.crypto.merlin.keystore.type=(keystore type – for example, jks)
org.apache.ws.security.crypto.merlin.keystore.password=(keystore password)
org.apache.ws.security.crypto.merlin.file=(keystore location – for example,
keystore/myStore.jks)
```

3.3.2 Fields and Methods

Constructor Summary:

```
public FS_Msg_Signer(java.lang.String QName,
                     java.lang.String URI)
```

Initializes security engines. The certificate alias and password must be set before a message can be signed.

Parameters:

QName – The element name of the application header that is to be signed – for example, "MeF" as in "m:MeF"

URI – The namespace URI for the application header that is to be signed – for example, "http://www.irs.gov/a2a/mef/MeFHeader.xsd" as in "xmlns:m = "http://www.irs.gov/a2a/mef/MeFHeader.xsd"

```
public FS_Msg_Signer(java.lang.String QName,
                     java.lang.String URI,
                     java.lang.String alias,
                     java.lang.StringBuffer password)
```

Initializes security engines.

Parameters:

QName – The element name of the application header that is to be signed – for example, "MeF" as in "m:MeF" or A2A as in "a:A2A"

URI – The namespace URI for the application header that is to be signed – for example, "http://www.irs.gov/a2a/mef/MeFHeader.xsd" as in *xmlns:m* = "http://www.irs.gov/a2a/mef/MeFHeader.xsd" or *xmlns:a*="http://www.irs.gov/a2a/A2AHeader.xsd"

alias – Alias to a certificate in a Java keystore that is to be used in signing

Methods Summary:

```
public org.apache.axis.Message sign(org.apache.axis.Message axisMessage)
```

Returns a signed Message using the WSS4J package from Apache.

Parameters:

axisMessage – Unsigned message

Returns:

Signed Message

```
public org.apache.axis.Message sign(org.apache.axis.Message axisMessage,
                                     java.lang.String alias,
                                     java.lang.StringBuffer password)
```

Returns a signed Message using the WSS4J package from Apache.

Parameters:

axisMessage – Unsigned message

alias – Alias to a certificate in a Java keystore that is to be used in signing

password – Password of the certificate to which the alias points

Returns:

Signed message

3.3.3 Exceptions

The FS_Msg_Signer message class deals with exception handling internally. Exceptions contain but are not limited to AxisFault, WSSecurityException, and SOAPException. Common issues are improperly formatted messages, incorrect references to signing parts, and attempts to access an already opened private key.

3.3.4 Integrating the Code

Integrating the Java signing code may require developers to change the signing codes package to match their project. Four external packages are required for the signing code to work:

Apache Axis 1.2.1 (axis.jar and saaj.jar)

Apache WSS4J (wss4j.jar)

Xerces 2.6.2 (xml-apis.jar)

Bouncy Castle Provider (bcprov-jdk14-146.jar) (132 was originally distributed, but is no longer available).

Newer versions may be used; however, some debugging may be required.

3.4 Visual Basic .Net Source

3.4.1 Class Description

The .Net implementation, FS_Msg_Signer.vb, contains a function that performs all procedures necessary for signing a SOAP message. This code was developed using the Microsoft Web Services Enhancement (WSE) 2.0 SP3 runtime libraries in a Visual Studio 2003 environment and follows the examples from <http://msdn.microsoft.com/library/default.asp?url=/library/en-us/wse/html/9421f0df-ccdd-4d67-9e85-1d102ada287e.asp>.

Certificates used for signing SOAP messages are contained within the Microsoft keystore.

3.4.2 Fields and Methods

Shared Function Summary:

```
public Shared Function sign(ByRef proxy As WebServicesClientProtocol,  
                             ByVal SigRef As String,  
                             ByVal KeyIdentifier As String)
```

Parameters:

proxy – WebServicesClientProtocol inherits SoapHttpClientProtocol; this object controls the communication between the client and the Web service

SigRef – ID value of the part that is to be signed

KeyIdentifier – String representation of the Subject Key Identifier from the signing certificate contained in the Microsoft keystore; remove all spaces from the character string

3.4.3 Integrating the Code

To use this code, first ensure that Microsoft WSE 2.0 SP3 is installed. Add FS_Msg_Signer.vb to your project's solution and add a reference to the WSE 2.0 SP3 runtime library (generally located at C:\Program Files\Microsoft WSE\v2.0\Microsoft.Web.Services2.dll after installation).

Calls to the signing code can be made as follows:

```
FS_Msg_Signer.Sign(_login, SigRef, KeyIdentifier)
```

Where _login is an object of the type WebservicesClientProtocol, SigRef is a string that contains the ID of the part of the soap message you wish to sign and KeyIdentifier is a string equal to the Subject Key Identifier field of your signing certificate with the spaces excluded.

If the client code has already been auto-generated for the Web service, a simple modification to the auto-generated code may need to be made. If the Web service function calls currently inherit System.Web.Services.Protocols.SoapHttpClientProtocol, replace all occurrences with Microsoft.Web.Services2.WebServicesClientProtocol.

Even though the signing code is written in Visual Basic it is interoperable with all .Net languages, including C#.

3.5 Design Considerations

Both the Java and .Net implementations of the signing code were developed using older environments. The code should have little trouble working with newer environments if all libraries and packages are updated. However, some calls may need to be changed with the use of newer libraries and packages.

3.5.1 Java Design Considerations

The Java signing code was designed using j2sdk1.4_13 and packages from Apache, particularly wss4j. The code as is should be compatible with newer versions of the Java sdk. However, it is recommended that SOAP messages be built from the ground up using `org.apache.axis.Message` instead of `javax.xml.soap.SOAPMessage`. The WSDL2Java tool is an excellent starting point for generating code stubs from WSDL.

3.5.2 .Net Design Considerations

The .Net code was designed in Visual Studio 2003 with WSE 2.0 SP3 from Microsoft. The code should work in a Visual Studio 2005 environment using WSE 3.0 from Microsoft. Minor adjustments may need to be made for calls to function contained in the WSE 3.0 runtime.

3.6 Authorized Certificate Authorities

Services at the IRS are tested to support X.509 v3-compatible certificates from the following Certificate Authorities (CA's) selected by the IRS for certificate issuance. Use of alternate CA's must be approved in advance by the IRS, since the certificate verification process includes verification of authenticity with the certificate issuer. Also, because of the verification requirements, the certificates must be issued by the Public side of the CA. Internal certificates, even from these CAs, will not be valid for use with IRS A2A Web services because IRS cannot validate them via the public CA.

Each of the listed certificates from any of these vendors will support the signature operation required in the A2A interface.

3.6.1 Entrust

While Entrust certificates associated with an Application Systems ID up through early 2012 are still accepted by the IRS, no newly issued Entrust certificates are accepted. Even certificates already accepted for one ASID are likely to be rejected for association with a new ASID.

3.6.2 Identrust

Identrust offers a "Business Representative" certificate at:

http://www.identrust.com/certificates/buy_aces_business.html, which is valid for use with the IRS. For users who wish their certificate to name a server rather than individual, the SSL certificate option, available at http://www.identrust.com/certificates/aces_step_by_step_ssl.html, also works properly.

3.6.3 Verisign (now operated by Symantec)

While Verisign certificates associated with an Application Systems ID up through early 2012 are still accepted by the IRS, no publicly offered Verisign certificates which meet the IRS' security requirements have passed interoperability testing with the IRS. If you have a PKI managed service with Verisign or Symantec, they may be able to issue some certificates for you which meet our current requirements as they have already done for one transmitter with such a service. Those Verisign certificates which were associated with an Applications System ID up through early 2012 are likely, like the Entrust ones, to be rejected for association with a new ASID.

3.6.4 Treasury Operational Certificate Authority

TOCA is used for internal IRS systems and some operated under contract with it. Application for such certificates requires you to have an internal IRS e-mail. [The current web site for preparing applications \(as of mid-2014\) is http://www.sems.enterprise.irs.gov/pkirequest/.](http://www.sems.enterprise.irs.gov/pkirequest/)

3.6.5 ORC

ORC's "component/server" certificates are available at <http://eca.orc.com>. As of August 2017, they do not offer business representative certificates.

Appendix A. The Signing Code

A.1 The Java Source Code

```
/**
 * Copyright (c) 2007, The MITRE Corporation
 * All rights reserved.
 *
 * Redistribution and use in source and binary forms, with or without modification, are permitted provided
 * that the following conditions are met:
 *
 * - Redistributions of source code must retain the above copyright notice, this list of conditions and
 * the following disclaimer.
 *
 * - Redistributions in binary form must reproduce the above copyright notice, this list of conditions
 * and the following disclaimer in the documentation and/or other materials provided with the
 * distribution.
 *
 * - Neither the name of the <ORGANIZATION> nor the names of its contributors may be used to endorse or
 * promote products derived from this software without specific prior written permission.
 *
 * THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND
 * ANY EXPRESS OR IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED
 * WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED.
 * IN NO EVENT SHALL THE COPYRIGHT OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT,
 * INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT
 * LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES; LOSS OF USE, DATA, OR PROFITS;
 * OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF LIABILITY, WHETHER IN
 * CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE) ARISING IN ANY
 * WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH DAMAGE.
 */

import org.apache.axis.Message;
import org.apache.axis.MessageContext;
import org.apache.axis.client.AxisClient;
import org.apache.axis.configuration.NullProvider;
```

```
30 import org.apache.axis.message.SOAPEnvelope;
31 import org.apache.axis.soap.SOAPConstants;
32 import org.apache.axis.AxisFault;
33 import org.apache.ws.axis.security.util.AxisUtil;
34 import org.apache.ws.security.components.crypto.Crypto;
35 import org.apache.ws.security.components.crypto.CryptoFactory;
36 import org.apache.ws.security.message.WSSignEnvelope;
37 import org.apache.ws.security.WSSecurityEngine;
38 import org.apache.ws.security.WSEncryptionPart;
39 import org.apache.ws.security.WSConstants;
40 import org.apache.ws.security.WSSecurityException;
41 import org.w3c.dom.Document;
42
43 import java.util.Vector;
44
45 import javax.xml.soap.SOAPException;
46
47
48 /**
49  * FS_Msg_Signer is a class designed to sign an unsigned SOAP Message using the WSS4J package from apache.
50  * Key information can be entered either by passing parameters or through properties files.
51  *
52  * The crypto.properties file contains information about the keystore and is formatted as shown:
53  *
54  * org.apache.ws.security.crypto.provider=org.apache.ws.security.components.crypto.Merlin
55  * org.apache.ws.security.crypto.merlin.keystore.type=(keystore type Example: jks)
56  * org.apache.ws.security.crypto.merlin.keystore.password=(key store password)
```

```
57  * org.apache.ws.security.crypto.merlin.file=(key store location Example: keystore/myKeyStore.jks)
58  * This code was written using j2sdk1.4.2_13. The following packages are also required for this
59  * code to work:
60  * <b>Apache Axis 1.2.1</b> - axis.jar and saaj.jar
61  * <b>Apache WSS4J</b> - wss4j.jar
62  * <b>Xerces 2.6.2</b> - xml-apis.jar
63  *
64  * @author Brian Henderson, The MITRE Corporation.
65  * @version 1.0
66  */
67  public class FS_Msg_Signer {
68
69      // The crypto.properties file contains all the information about the key store and the crypto engine.
70      // it is located in the base directory
71      private final Crypto crypto;
72
73      // Though secEngine is never used it is necessary because it initializes the security engine that is
74      // used for signing.
75      private final WSSecurityEngine secEngine;
76
77      private final AxisClient engine;
78      private final MessageContext msgContext;
79
80      private final WSSignEnvelope signer;
81      private final String QName;
82      private final String URI;
83
```

```
84      /**
85       * Initializes security engines. The certificate alias and password must be set before a
86       * message can be signed
87       *
88       * @param QName          First part of the reference to the message part that is to be signed.
89       * Example: "MeF" as in "m:MeF"
90       * @param URI            Second part of the reference to the message part that is to be signed.
91       * Example: "http://prime.irs.gov/a2a/pki/MeFHeader.xsd" as in
92       * "xmlns:m = "http://prime.irs.gov/a2a/pki/MeFHeader.xsd"
93       */
94     public FS_Msg_Signer(String QName, String URI){
95         crypto = CryptoFactory.getInstance("crypto.properties");
96         secEngine = new WSSecurityEngine();
97
98         signer = new WSSignEnvelope();
99
100         this.QName = QName;
101         this.URI = URI;
102     }
103
104     /**
105     * Initializes security engines.
106     *
107     * @param QName          First part of the reference to the message part that is to be signed.
108     * Example: "MeF" as in "m:MeF"
109     * @param URI            Second part of the reference to the message part that is to be signed.
110     * Example: "http://prime.irs.gov/a2a/pki/MeFHeader.xsd" as in
```

```
111     * "xmlns:m = "http://prime.irs.gov/a2a/pki/MeFHeader.xsd"
112     * @param alias      Alias to a certificate in a java key store that is to be used in signing.
113     * @param password    Password of the certificate that the alias points to.
114     */
115     public FS_Msg_Signer(String QName, String URI, String alias, StringBuffer password){
116         crypto = CryptoFactory.getInstance("crypto.properties");
117         secEngine = new WSSecurityEngine();
118
119         signer = new WSSignEnvelope();
120
121         signer.setUserInfo(alias, password.toString());
122         password.replace( 0, password.length(), "*****");
123
124         this.QName = QName;
125         this.URI = URI;
126     }
127
128     /**
129     * Returns a signed Message using the WSS4J package from apache.
130     *
131     * @param axisMessage    Unsigned message.
132     * @param alias      Alias to a certificate in a java key store that is to be used in signing.
133     * @param password    Password of the certificate that the alias points to.
134     * @return Signed Message.
135     */
136     public Message sign (final Message axisMessage, final String alias, final StringBuffer password){
137         engine = new AxisClient(new NullProvider());
```

```
138         msgContext = new MessageContext(engine);
139
140         axisMessage.setMessageContext(msgContext);
141
142         Message signedSOAPMsg = null;
143
144         try{
145             //Extract the SOAPEnvelope from the Message
146             final SOAPEnvelope unsignedEnvelope = axisMessage.getSOAPEnvelope();
147
148             //Extract the SOAPConstants from the SOAPEnvelope
149             final SOAPConstants soapConstants = unsignedEnvelope.getSOAPConstants();
150
151             //Create a new envelope with WS security features
152             final WSSignEnvelope signer2 = new WSSignEnvelope();
153
154             //Set the user info for the signer (determines which certificate to use)
155             //The alias is the alias to the private key you want to use in your java key store
156             //The password is the private key password
157             signer2.setUserInfo(alias, password.toString());
158             password.replace( 0, password.length(), "*****");
159
160             //By default the binary security token is not added to the header
161             //This line tells the signer to add the binary security token
162             signer2.setKeyIdentifierType(WSConstants.BST_DIRECT_REFERENCE);
163
164             //Set wsse:Security MustUnderstand to "0"
```

```
165         signer2.setMustUnderstand(false);
166
167         //This block adds the various parts that we want signed
168         //In this case the MeF header and the Body
169         //The first value passed to the WSEncryptionPart constructor is the QName
170         //for the MeF header it is "MeF", for the body soapConstants.getBodyQName() = "Body"
171         //The second value passed to the WSEncryptionPart constructor is the name space (xmlns:)
172         //for the MeF header it is "http://www.irs.gov/a2a/mef/MeFHeader.xsd",
173         //for the body soapConstants.getEnvelopeURI() =
174         //"http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd"
175         final Vector parts = new Vector();
176         final WSEncryptionPart part1 = new WSEncryptionPart(QName, URI, "Content");
177         final WSEncryptionPart part2 = new
178             WSEncryptionPart(soapConstants.getBodyQName().getLocalPart(),
179                 soapConstants.getEnvelopeURI(), "Content");
180
181         parts.add(part1);
182         parts.add(part2);
183         signer2.setParts(parts);
184
185         //Grabs the SOAPEnvelope in org.w3c.dom.Document form
186         final Document doc = unsignedEnvelope.getAsDocument();
187
188         //The document and the crypto properties are passed to the signer to build the signed message
189         final Document signedDoc = signer2.build(doc, crypto);
190
191         // Convert the signed document into a SOAP message.
```

```
192         signedSOAPMsg = (org.apache.axis.Message)AxisUtil.toSOAPMessage(signedDoc);
193
194         // Adds the XML declaration, necessary for parsing and understanding the message on the
195         // receiving side
196         signedSOAPMsg.setProperty(Message.WRITE_XML_DECLARATION, "true");
197     }catch(AxisFault af){
198         System.out.println(af.toString());
199     }catch(WSSecurityException wsse){
200         System.out.println(wsse.toString());
201         System.out.println("Possibly trying to access an already open certificate");
202     }catch(SOAPException se){
203         System.out.println(se.toString());
204     }catch(Exception e){
205         System.out.println(e.toString());
206     }
207     return signedSOAPMsg;
208 }
209
210 /**
211  * Returns a signed Message using the WSS4J package from apache.
212  *
213  * @param axisMessage Unsigned message.
214  * @return Signed Message.
215  */
216 public Message sign (final Message axisMessage){
217     engine = new AxisClient(new NullProvider());
218     msgContext = new MessageContext(engine);
```

```
219
220     axisMessage.setMessageContext(msgContext);
221
222     Message signedSOAPMsg = null;
223
224     try{
225         //Extract the SOAPEnvelope from the Message
226         final SOAPEnvelope unsignedEnvelope = axisMessage.getSOAPEnvelope();
227
228         //Extract the SOAPConstants from the SOAPEnvelope
229         final SOAPConstants soapConstants = unsignedEnvelope.getSOAPConstants();
230
231         //By default the binary security token is not added to the header
232         //This line tells the signer to add the binary security token
233         signer.setKeyIdentifierType(WSConstants.BST_DIRECT_REFERENCE);
234
235         //Set wsse:Security MustUnderstand to "0"
236         signer.setMustUnderstand(false);
237
238         //This block adds the various parts that we want signed
239         //In this case the MeF header and the Body
240         //The first value passed to the WSEncryptionPart constructor is the QName
241         //for the MeF header it is "MeF", for the body soapConstants.getBodyQName() = "Body"
242         //The second value passed to the WSEncryptionPart constructor is the name space (xmlns:)
243         //for the MeF header it is "http://www.irs.gov/a2a/mef/MeFHeader.xsd", for the body
244         //soapConstants.getEnvelopeURI() =
245         //"http://docs.oasis-open.org/wss/2004/01/oasis-200401-wss-wssecurity-utility-1.0.xsd"
```

```
246         final Vector parts = new Vector();
247         final WSEncryptionPart part1 = new WSEncryptionPart(QName, URI, "Content");
248         final WSEncryptionPart part2 = new
249             WSEncryptionPart(soapConstants.getBodyQName().getLocalPart(),
250             soapConstants.getEnvelopeURI(), "Content");
251
252         parts.add(part1);
253         parts.add(part2);
254         signer.setParts(parts);
255
256         //Grabs the SOAPEnvelope in org.w3c.dom.Document form
257         final Document doc = unsignedEnvelope.getAsDocument();
258
259         //The document and the crypto properties are passed to the signer to build the signed message
260         final Document signedDoc = signer.build(doc, crypto);
261
262         // Convert the signed document into a SOAP message.
263         signedSOAPMsg = (org.apache.axis.Message)AxisUtil.toSOAPMessage(signedDoc);
264         // Adds the XML declaration, necessary for parsing and understanding the message on
265         // the receiving side
266         signedSOAPMsg.setProperty(Message.WRITE_XML_DECLARATION, "true");
267     }catch(AxisFault af){
268         System.out.println(af.toString());
269     }catch(WSSecurityException wsse){
270         System.out.println(wsse.toString());
271         System.out.println("Possibly trying to access an already open certificate");
272     }catch(SOAPException se){
```

```
273         System.out.println(se.toString());
274     }catch(Exception e){
275         System.out.println(e.toString());
276         System.out.println("Most likely you did not set the alias and password");
277     }
278     return signedSOAPMsg;
279 }
280 }
```

1 A.2 The .Net Source Code

```
2 'Copyright (c) 2007, The MITRE Corporation
3 '    All rights reserved.
4 'Redistribution and use in source and binary forms, with or without modification, are permitted
5 'provided that the following conditions are met:
6 '    - Redistributions of source code must retain the above copyright notice, this list of
7 'conditions and the following disclaimer.
8 '    - Redistributions in binary form must reproduce the above copyright notice, this list of
9 '    conditions and the following disclaimer in the documentation and/or other materials
10 '    provided with the distribution.
11 '    - Neither the name of the <ORGANIZATION> nor the names of its contributors may be used to
12 '    endorse or promote products derived from this software without specific prior written permission.
13 '
14 'THIS SOFTWARE IS PROVIDED BY THE COPYRIGHT HOLDERS AND CONTRIBUTORS "AS IS" AND ANY EXPRESS OR
15 'IMPLIED WARRANTIES, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY
16 'AND FITNESS FOR A PARTICULAR PURPOSE ARE DISCLAIMED. IN NO EVENT SHALL THE COPYRIGHT
17 'OWNER OR CONTRIBUTORS BE LIABLE FOR ANY DIRECT, INDIRECT, INCIDENTAL, SPECIAL, EXEMPLARY, OR
18 'CONSEQUENTIAL DAMAGES (INCLUDING, BUT NOT LIMITED TO, PROCUREMENT OF SUBSTITUTE GOODS OR SERVICES;
```

```
19 'LOSS OF USE, DATA, OR PROFITS; OR BUSINESS INTERRUPTION) HOWEVER CAUSED AND ON ANY THEORY OF
20 'LIABILITY, WHETHER IN CONTRACT, STRICT LIABILITY, OR TORT (INCLUDING NEGLIGENCE OR OTHERWISE)
21 'ARISING IN ANY WAY OUT OF THE USE OF THIS SOFTWARE, EVEN IF ADVISED OF THE POSSIBILITY OF SUCH
22 'DAMAGE.
23 '
24 'THIS SOFTWARE IS A SAMPLE PROTOTYPE AND IS NOT INTENDED FOR DIRECT USE IN A PRODUCTION ENVIRONMENT WITHOUT
25 FURTHER DEVELOPMENT AND THOROUGH TESTING.
26
27 Imports Microsoft.Web.Services2
28 Imports Microsoft.Web.Services2.Security
29 Imports Microsoft.Web.Services2.Security.Tokens
30 Imports Microsoft.Web.Services2.Security.X509
31 Imports Microsoft.Web.Services2.Dime
32 Imports System.Configuration
33
34 '' FS_Msg_Signer is a class designed to sign an unsigned SOAP Message using the Microsoft
35 ' WSE 2.0 SP3 runtime libraries.
36 '
37 ' @author Brian Henderson, The MITRE Corporation.
38 ' @version 1.0
39
40 Public Class FS_Msg_Signer
41
42     '' This Function retrieves the X509SecurityToken from the certificate store
43     ' Here we are using the Microsoft CAPI as the certificate store
44     '
45     ' @param KeyIdentifier      String representation of the Subject Key Identifier from the
```

```
46      ' signing certificate contained in the Microsoft key store. (Remove any spaces or delimiters)
47      ' @return X509SecurityToken a binary representation of the certificate retrieved from the
48      ' certificate store.
49      '
50      ' Example taken from http://msdn.microsoft.com/library/default.asp?url=
51      ' /library/en-us/wse/html/9421f0df-ccdd-4d67-9e85-1d102ada287e.asp
52
53      Private Shared Function GetSigningToken(ByVal keyIdentifier As String)
54          Dim token As X509SecurityToken
55          Dim store As X509CertificateStore
56          token = Nothing
57          store = Nothing
58          'opens certificate store
59          store = X509CertificateStore.CurrentUserStore(X509CertificateStore.MyStore)
60          If Not store.OpenRead Then
61              Return Nothing
62          End If
63
64          Dim certs As X509CertificateCollection
65          'pulls out all certificates that have this particular CertKeyIdentifier
66          certs = store.FindCertificateByKeyIdentifier(ConvertHexToByteArray(keyIdentifier))
67
68          Dim certEnum As IEnumerator
69          certEnum = certs.GetEnumerator
70          certEnum.MoveNext()
71
72          'creates the X509SecurityToken
```

```
73         If certs.Count > 0 Then
74             token = New X509SecurityToken(DirectCast(certEnum.Current, X509Certificate))
75         End If
76
77         'returns the X509SecurityToken
78         Return token
79
80     End Function
81
82     '' This function has been added to convert a string representation of a hexadecimal value
83     ' to a byte array
84     ' Currently it is assumed that the input is correct there is no error checking or correction.
85     '
86     ' @param hexString  a string of hexadecimal values with no delimiters(spaces) in between each value
87     ' @return Byte      a byte array representation of the hexadecimal string
88     '
89     ' This function was derived from the examples at http://www.codeproject.com/csharp/hexencoding.asp
90
91     Private Shared Function ConvertHexToByteArray(ByVal hexString As String)
92         Dim i As Integer
93         Dim j As Integer
94         Dim bytes((hexString.Length / 2) - 1) As Byte
95         Dim hex As String
96         Dim hexCharArray() As Char
97
98         hexCharArray = hexString.ToCharArray
99         j = 0
```

```
100
101     For i = 0 To (bytes.Length - 1)
102         hex = New String(New Char() {hexCharArray(j), hexCharArray(j + 1)})
103         bytes(i) = Byte.Parse(hex, System.Globalization.NumberStyles.HexNumber)
104         j = j + 2
105     Next
106     Return bytes
107 End Function

108
109 ''Preforms operations to sign a soap message
110 '
111 ' @param proxy          WebServicesClientProtocol (this type require Microsoft WSE 2.0 SP3
112 ' runtime to be installed)
113 ' @param sigRef         String reference to the ID of the part that is to be signed
114 ' @param keyIdentifier  String representation of the Subject Key Identifier from the signing
115 ' certificate contained in the Microsoft key store. (Remove any spaces)
116 ' @return No Return     However, proxy is passed in by reference and operations are made upon it.
117 '
118 ' Example taken from http://msdn.microsoft.com/library/default.asp?url=
119 ' /library/en-us/wse/html/9421f0df-ccdd-4d67-9e85-1d102ada287e.asp
120 Public Shared Function Sign(ByRef proxy As WebServicesClientProtocol, ByVal sigRef As String,
121                             ByVal keyIdentifier As String)
122
123     Dim token As X509SecurityToken
124     Dim sig As MessageSignature
125
126     'This is the reference to the MeF header. The value here is the new MeF.ID in MeFStateServiceWse.vb
```

```
127         Dim soapRef As New SignatureReference(sigRef)
128
129         'Canonicalizes the MeF header
130         soapRef.AddTransform(New Microsoft.Web.Services2.Security.Xml.XmlDsigExcC14NTransform)
131
132         'Calls the function GetSigningToken to retrieve the X509SecurityToken from the store
133         token = GetSigningToken(keyIdentifier)
134
135         'Adds the X509SecurityToken to the soap header
136         proxy.RequestSoapContext.Security.Tokens.Add(token)
137
138         'Creates a MessageSignature object from the X509SecurityToken
139         sig = New MessageSignature(token)
140
141         'Clears the elements to be signed
142         sig.SignatureOptions = SignatureOptions.IncludeNone
143
144         'Adds the soap body to the list of elements to be signed
145         sig.SignatureOptions = SignatureOptions.IncludeSoapBody
146
147         'Adds the MeF header to the list of elements to be signed
148         sig.AddReference(soapRef)
149
150         'Adds the MessageSignature to the soap header
151         proxy.RequestSoapContext.Security.Elements.Add(sig)
152
153     End Function
```

154 [End Class](#)

Acronyms

A2A	Application to Application
ASID	Application System ID
CA	Certificate Authority
CRL	Certificate Revocation List
DNS	Domain Name Server
IATF	Information Assurance Technical Framework
IRS	Internal Revenue Service
MeF	Modernized e-File
PKI	Public Key Infrastructure
SOAP	Simple Object Access Protocol
TIGERS	Tax Information Group for Electronic Data Interchange Requirements Standardization
TTP	Trusted Third Party
WS	Web Services
WSDL	Web Services Design Language
WSE	Web Services Enhancement
XML	eXtensible Markup Language

Glossary

Authentication	A security measure designed to establish the validity of a transmission, message, or originator, or a means of verifying an individual's authorization to receive specific categories of information.
Certificate (aka “digital certificate”)	A digital representation of information that, at a minimum, (1) identifies the certification authority issuing it, (2) names or identifies its subscriber, (3) contains the subscriber's public key, (4) identifies its operational period, and (5) is digitally signed by the certification authority issuing it.
Certificate Authority (CA)	An authority trusted by one or more users to issue and manage X.509.
Client-Side Certificate	A X.509-compatible digital certificate used by an end user computer system when establishing a Secure Socket Layer session with a server at the IRS.
Digital Signature	The result of a transformation of a message by means of a cryptographic system using keys such that a Relying Party can determine (1) whether the transformation was created using the private key that corresponds to the public key in the signer's digital certificate and (2) whether the message has been altered since the transformation was made.
External	A state, transmitter, Electronic Return Originator, or organization not under the direct control of the IRS that participates in the Modernized eFile A2A Web services program.
PKI Sponsor	Fills the role of a Subscriber for non-human system components that are named as public key certificate subjects. The PKI Sponsor is responsible for meeting the obligations of Subscribers as defined throughout this document.
Private Key	(1) The key of a signature key pair used to create a digital signature. (2) The key of an encryption key pair used to decrypt confidential information. In both cases, this key must be kept secret.
Public Key	(1) The key of a signature key pair used to validate a digital signature. (2) The key of an encryption key pair used to encrypt confidential information. In both cases, this key is made publicly available, normally in the form of a digital certificate.
SOAP Message	Simple Object Access Protocol structured data item used in a Web Services environment to pass information from one cooperating system to another.

References

1. Homeland Security Presidential Directive 12: Policy for a Common Identification Standard for Federal Employees and Contractors
2. ISO X.509 Digital Certificate Standard
3. Java Web Services Security Signing Implementation: <http://ws.apache.org/wss4j/api.html>
4. Microsoft Brokered Authentication with message signing: <http://msdn2.microsoft.com/en-us/library/ms824642.aspx>
5. National Institute of Standards and Technology Crypto Enabled Applications: <http://csrc.nist.gov/pki/crypto-appl.html>
6. W3C Web Services Security Standard 1.0
7. XML Canonicalization Standard RFC 3076
8. XML Signature Standard RFC 3275